

AI-Native Self-Healing O-RAN Network using a Digital Twin

Per-slide explanation, delivery notes, terminology and anticipated questions

Companion to **ORAN_PRISM_Review.pptx** — 18 slides (12 presented, 6 appendix)
Target: 8–10 minute Samsung PRISM worklet review · 17 August 2026

How to use this. One page per slide. **What to say** is the delivery line, not a script — say it in your own words. **Terms** explains every piece of jargon on that slide, so you are never caught defining something mid-answer. **If asked** gives the answer to the question that slide most invites, including the ones where the honest answer is a concession.

Every figure traces to `ORAN_Project_Results.xlsx`, `README.md`, `TEAM_REPORT.md` or `ORAN_SPlane_Technical_Report.pdf`. Nothing here was invented for the deck.

SAMSUNG PRISM · WORKLET REVIEW

AI-Native Self-Healing O-RAN Network using a Digital Twin

Open Fronthaul Synchronisation-Plane timing security — telling a benign timing fault from a deliberate attack, and committing a governed recovery inside the 2-second failure window.

Software complete

56 tests passing

commit 4e4bd16

Tier-3 hardware: not started

Students Tanmaya Kumar · Raghu Ram K · Muniapalle Jaswanth Kumar

Mentors Bikas Singh (NovaThink Tech) · Navin Kumar (Amrita Vishwa Vidyapeetham)

17 August 2026

Cover: project name, one-line scope, status chips, team, date.

WHAT TO SAY

- Open with the one-liner: we built and validated, on real 5G timing data, a system that separates timing faults from attacks and heals inside the 2-second deadline — and fails safely when it cannot trust its own data.
- Point at the four status chips and stop. They pre-empt the two questions everyone asks: is it finished, and is there hardware. Answer: software yes, hardware no.
- Do not read the team list aloud.

TERMS ON THIS SLIDE

S-plane

The part of the Open Fronthaul link that carries clock traffic. 'S' is for synchronisation. This is the whole topic of the project.

Open Fronthaul

The link between the radio unit (O-RU) and the baseband unit (O-DU) in a disaggregated 5G base station.

Governed recovery

A fix that is only applied after passing explicit safety checks, rather than fired automatically on an alarm.

commit 4e4bd16

The exact git commit every number in this deck was produced from. Quoting it is what makes the results checkable.

IF ASKED

Is this finished? Software is complete and validated. Tier-3 hardware validation has not started — that is the ask.

2 Two numbers define the whole problem

THE PROBLEM

Two numbers define the whole problem

100 ns

Accuracy to which every 5G radio on the fronthaul must agree on time

2 s

Time from timing failure to service failure — the whole decision budget

PTP (IEEE 1588) carries the clock and has no built-in security.

A faulty or malicious clock can take a base station offline. The hard part is not detection — it is **attribution**.

The symptom is identical. The correct response is opposite.

Benign fault (H0)	Malicious attack (H1)
GPS signal lost — clock rides its internal backup	GPS spoofed — clock believes a fake time
Network congestion adds delay and jitter	Attacker injects one-sided delay on purpose
A link fails and quality drops	Attacker forges 'I am the best clock' messages
A planned, legitimate switch-over	A rogue device impersonating the master

For a fault, wait it out — the real clock returns. For an attack, isolate the source and switch away.

The central claim. A system that only shouts "something is wrong" leaves the operator with a coin-flip under a 2-second deadline. Telling H0 from H1 is what makes an automatic response safe enough to attempt at all.

The 100 ns and 2 s budgets, the PTP security gap, and a four-row table of H0 vs H1 look-alikes.

WHAT TO SAY

- Spend real time here. This slide is the justification for the entire project.
- Say the numbers slowly: radios must agree on time to about 100 nanoseconds, and timing failure becomes service failure in about 2 seconds.
- Then the table: read one row across. 'GPS signal lost' and 'GPS spoofed' produce the same symptom and demand opposite responses.
- Land the central claim verbatim — an alarm-only system leaves the operator with a coin-flip under a 2-second deadline.

TERMS ON THIS SLIDE

100 ns time-error budget

Maximum tolerable deviation from correct time. One ten-millionth of a second; roughly the time light travels 30 metres. Beyond it, radio performance degrades and neighbouring cells interfere.

Why radios need it

TDD (uplink and downlink share one frequency, separated only in time), carrier aggregation and MIMO (combining antennas requires known relative phase), and handover (both cells need a shared timesbase).

2 s failure window

Published time from the onset of a timing attack to base-station service failure. The entire budget for noticing, deciding and acting.

PTP (IEEE 1588)

Precision Time Protocol. Distributes actual time-of-day across the network via a four-timestamp message exchange. Has no built-in authentication.

H0 / H1

Statistical notation borrowed for the two labels. H0 = benign fault. H1 = malicious attack.

Holdover

Losing the reference and free-running on the local oscillator, drifting slowly. A normal, benign condition — and exactly what a GNSS attack is

designed to imitate.

IF ASKED

Where does 100 ns come from? KNOWN GAP: your own documents state '~100 ns for O-RAN fronthaul' without citing a clause. It is most likely a fronthaul-segment allocation out of the 3GPP air-interface Time Alignment Error budget, or the tighter MIMO class — not the end-to-end figure. Find the exact citation before the review; this is an obvious opening question.

Why can't an operator just do this manually? Two seconds. There is no human-in-the-loop option at that timescale.

APPROVED WORKLET

What we committed to

AI-Native Self-Healing Open-Radio Access Network (O-RAN) Network using Digital Twin 

AI Native Self-Healing O-RAN Network using Digital Twin proposes an autonomous network management framework that combines AI-based anomaly detection with a digital twin to predict, validate, and resolve RAN faults in a closed loop. The project targets improved reliability, faster recovery, and lower operational effort for future O-RAN deployments.

Problem Statement
O-RAN networks are becoming more complex and distributed, which makes manual fault diagnosis and recovery slower and less reliable. Traditional static alarms or rule-based responses often react too late, require human intervention, and may not identify the true root cause of a problem. The project aims to solve this by combining AI and a digital twin to create a closed-loop self-healing mechanism that can sense, predict, decide, and verify recovery actions.

Worklet objective

- Detect anomalies in O-RAN KPIs using AI-based monitoring.
- Predict faults before they cause major service disruption.
- Use a digital twin to simulate network conditions and validate healing actions safely.
- Automate corrective actions such as parameter tuning, rerouting, or recovery workflows.
- Compare the AI-native approach with traditional fault management methods.

References
Research on **Closed-Loop Autonomous Network Healing Using Digital Twins**, which directly matches the project theme. The overview of **Digital Twin for O-RAN Towards 6G**, which explains why digital twins are useful for open and autonomous RANs.
Material on **AI in O-RAN** describing predictive maintenance, anomaly detection, and self-healing capabilities.
Work on **anomaly detection in Open RAN** showing how digital twins support controlled KPI streams, repeatable fault injection, and model training.
Broader AI-native autonomy discussions for O-RAN and 6G evolution.
<https://www.3gpp.org/technologies/5g-system-overview> (BRIEF)

Core idea
Build a closed-loop system where a digital twin mirrors the live O-RAN environment, detects anomalies, predicts faults, and recommends or triggers healing actions before service impact grows. Research on O-RAN digital twins and AI-driven self-healing highlights this as a practical path toward autonomous RAN operations.

Scope of work
The work should cover anomaly detection, root-cause analysis, fault prediction, and automated recovery in an O-RAN environment. The digital twin can be used to emulate network states, inject faults, test mitigation actions safely, and train AI models using realistic KPI streams. A strong PRISM scope also includes comparison against rule-based healing and evaluation under different fault scenarios such as configuration errors, traffic overload, and node/link degradation.

Expected Outcome

- A conceptual architecture for AI-native self-healing in O-RAN.
- A digital-twin-based testbed for fault injection and recovery validation.
- AI models for anomaly detection or predictive fault identification.
- Faster fault recovery and reduced downtime compared with manual or rule-based methods.
- Evaluation results showing improvements in availability, recovery time, and operational efficiency.

M1: 0-8 days	M2, & M3 (10-22 days)	M4, M5, M6 (24-36 days)	M6 (40-50 days)
Problem study and Literature Survey Define O-RAN self-healing requirements and digital twin use cases. System Architecture design Finalize data flow between live O-RAN, AI engine, and digital twin.	M2: KPI and fault taxonomy Identify KPIs, anomaly types, and healing targets. M3: Digital twin setup Build or simulate the O-RAN twin and fault injection environment.	M4: AI model development Implement anomaly detection or predictive fault models. M5: Healing logic integration Add decision engine for automated or assisted recovery actions. M6: Testing and evaluation Measure detection accuracy, recovery time, and service continuity.	M6: Paper & IP Draft (Publication + Patent disclosure) Final documentation Prepare results, findings, and future enhancement ideas

Students:

Tanmaya Kumar	bl.en.u4eac23058@bl.students.amrita.edu
Raghu Ram K	bl.sc.u4cse24142@bl.students.amrita.edu
Munipalle Jaswanth Kumar	bl.sc.u4cse24130@bl.students.amrita.edu

Mentors:

Bikas Singh (NovaThink Tech)
Navin Kumar (k. Navin@blr.amrita.edu)
9019930325

The approved Samsung PRISM worklet one-pager, full size.

WHAT TO SAY

- Do not read this slide. Point at it, say 'this is what was approved', and move to the next slide, which maps every objective to what shipped.
- It is here so nobody can claim scope drift.

TERMS ON THIS SLIDE

Worklet

Samsung PRISM's term for the defined unit of project scope you signed up to deliver.

M1-M6

The milestone timeline along the bottom: problem study and architecture, KPI/fault taxonomy and twin setup, AI model and healing logic, testing, then paper and IP draft.

IF ASKED

Did scope change? No. The next slide maps all five stated objectives to shipped artefacts with measured evidence.

COMMITTED → DELIVERED

Every worklet objective, and what shipped against it

Worklet objective	What was built	Measured evidence
Detect anomalies in O-RAN KPIs using AI-based monitoring	28 features in 6 groups → Random Forest H0/H1, plus a group-wise Isolation Forest open-set layer	0.991 ± 0.002 accuracy; ROC-AUC 1.000 ± 0.000 (8 seeds)
Predict faults before they cause major service disruption	0.4 s windows stepped every 0.2 s; 2-of-3 persistence on the UNKNOWN and H1 channels	Live decision latency 0.081 s mean / 0.404 s max
Use a digital twin to validate healing actions safely	Analytical forward model projecting time error over a 1 s horizon, with a fidelity (trust) score	Pearson $r = 0.998$ against simulator physics; fidelity < 0.35 forces the safe default
Automate corrective actions (tuning, rerouting, recovery)	Six governed actions: switch timing source (LLS-C1/C2/C3), GNSS failover, holdover, isolate rogue master, reroute, safe default	Recovery 1.000; wrong-action 0.000; MTTR 0.933 ± 0.008 s
Compare the AI-native approach with traditional fault management	Benchmark against always-failover, always-holdover and detection-only baselines	Only the governed loop reaches 1.00 recovery with 0.00 wrong-action

Sources: README.md · TEAM_REPORT.md · ORAN_Project_Results.xlsx (3_Results_Core, 5_Real_and_Live) · ORAN_SPlane_Technical_Report.pdf §3.2

Five-row table: objective, what was built, measured evidence.

WHAT TO SAY

- This is the accountability slide. Walk down the right-hand column only — the numbers.
- Say explicitly that every figure in that column traces to a CSV on disk under results/.
- Expect to be stopped here. It is the natural place for a mentor to interrupt.

TERMS ON THIS SLIDE

28 features in 6 groups	Every 0.4 s the raw clock traffic is reduced to 28 measured numbers, sorted into six categories.
Random Forest	A classifier made of many decision trees that vote. Each tree asks yes/no questions about the 28 numbers. Not deep learning.
Isolation Forest	A different algorithm answering a different question: not 'which category is this?' but 'have I ever seen anything like it?' It flags outliers.
Open-set layer	The ability to answer 'none of the above'. A closed-set classifier is forced to pick a known category, so a brand-new attack gets crammed into the wrong box.
Group-wise	One Isolation Forest per feature group rather than one over all 28. Matters because a replay attack is strange in timing space but perfectly ordinary in BMCA space — a single detector averages the signal away.
ROC-AUC 1.000	How separable the two classes are, independent of where the decision threshold sits. 0.5 = coin flip, 1.0 = perfectly separable.
8 seeds	A seed fixes the random number generator. Eight runs with different seeds proves the result is not one lucky run.

2-of-3 persistence	Require 2 of the last 3 windows to agree before acting. Kills single-window noise at a cost of ~0.4 s.
Analytical forward model	The digital twin is a physics equation, not a trained model. Given current clock error and a proposed action, it computes how the error evolves.
Fidelity score	The twin rating its own trustworthiness. Below 0.35 the system stops believing its own forecast and takes the safe default.
Pearson $r = 0.998$	Correlation between the twin's prediction and what the simulator actually did. Validates that it ranks actions in the right order.
MTTR	Mean Time To Recovery — average time from fault onset to timing back in spec.
LLS-C1/C2/C3	O-RAN's standard names for fronthaul synchronisation topologies — different arrangements of where the clock source sits relative to the O-RU and O-DU.

IF ASKED

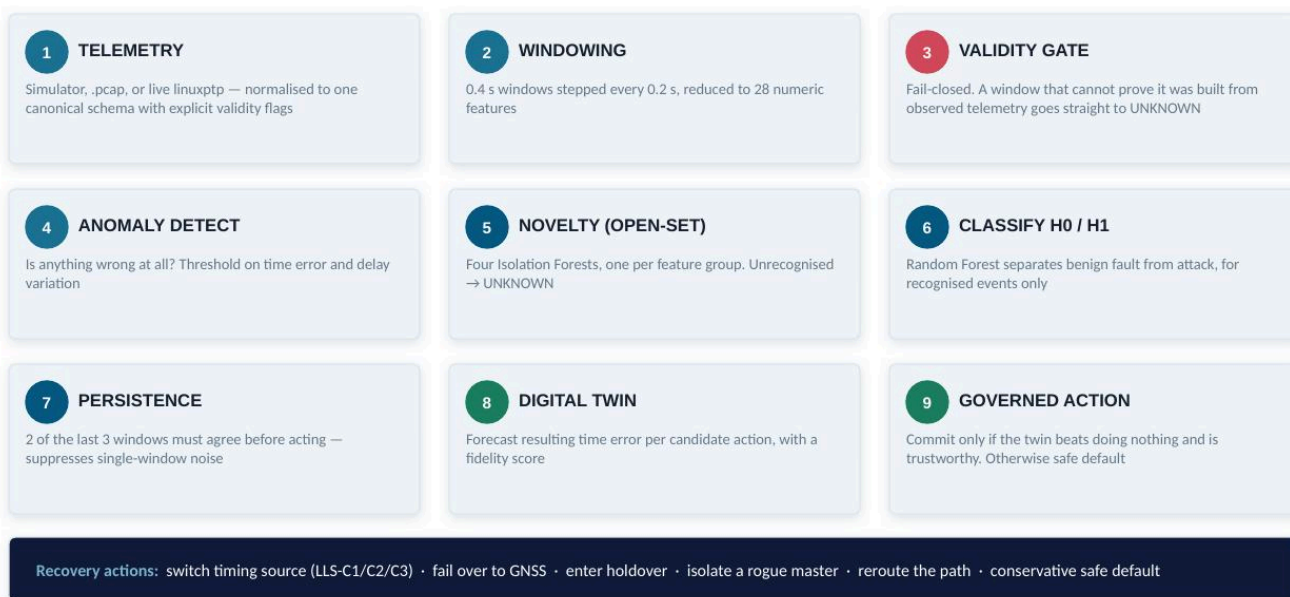
ROC-AUC 1.000 and 99.1% accuracy together — is that leakage? They are consistent: AUC measures separability across all thresholds, accuracy measures one operating point. Volunteer the leakage defence before being asked: raw grandmaster identities and MAC addresses were deliberately excluded as features precisely so the model could not fingerprint the capture file.

Does it really predict faults? Be careful. It detects fast (0.081 s), it does not forecast a fault that has not happened. The twin forecasts the consequence of a FIX, not the arrival of a fault. Say that it meets the operational intent and that true predictive lead time needs the longer baselines in the Tier-3 plan.

Why Random Forest and not gradient boosting? Honest answer: no measured model comparison exists in the documentation. The defensible line is that simulator ROC-AUC is already 1.000 and real-data protection is 99.96%, so the ceiling is the data, not the model — a different classifier has no headroom to show an advantage. Running one LightGBM baseline before the review would convert this from an argument into a measurement.

SYSTEM ARCHITECTURE

A sequential pipeline where every stage can veto



Nine numbered stages in a 3x3 grid, plus the six recovery actions along the bottom.

WHAT TO SAY

- Do not narrate all nine. Name the shape — telemetry in at stage 1, a governed action out at stage 9 — then stop on two stages.
- Stage 3, the fail-closed validity gate: it runs BEFORE any model. That ordering is the single most important design decision in the system and it came out of a real bug (slide 10).
- Stage 5, group-wise novelty: explain why four detectors beat one.
- Read the recovery action list once so the audience knows the output is a concrete action, not a score.

TERMS ON THIS SLIDE

Canonical schema

One common data format that the simulator, pcap files and live linuxptp all get translated into, so the rest of the pipeline never has to care where data came from. This is what made real-data validation possible at all.

0.4 s window, 0.2 s step

Slices overlap by half, so a fresh decision arrives every 0.2 s instead of every 0.4 s — faster reaction without shortening the window.

Fail-closed

Refusing to certify 'healthy' when the data cannot be trusted. The opposite, fail-open, is what the bug on slide 10 was.

Weighted Sidak correction

A statistical adjustment applied per group so that combining four detectors keeps the total false-alarm rate inside a configured 2% budget, rather than multiplying it by four.

Isolate a rogue master

Block the device claiming to be the best clock. Correct against a BMCA attack, wrong against a legitimate failover — which is exactly why H0/H1 discrimination matters.

Reroute

Send clock traffic over a different network path.

IF ASKED

What if two stages disagree? The pipeline is conservative by construction: any stage can veto to UNKNOWN, and UNKNOWN routes to the safe default. There is no voting between stages, only successive gates.

DATA PROVENANCE

Three data sources, deliberately unequal in weight

A

Our simulator

SYNTHETIC

Physics-based PTP clock with perfectly labelled benign faults and attacks. Our sandbox for building and debugging.

Weakest evidence. Most large simulator percentages come from here.

B

TIMESAFE public captures

REAL ATTACK

Real PTP recordings from a 5G testbed at Northeastern University, published with their research. Announce, Sync/Follow-Up and single-step attacks, each with a benign baseline.

5 independent sessions - ~4.5k-47k rows each. This is what the ~99% claims rest on.

C

Our own live netem runs

REAL-LIVE

Two real ptp4l daemons over a virtual link with injected impairments — baseline, delay variation, packet loss, master outage.

5,665 real-trace telemetry windows across four scenarios.

Honest limit, stated up front. Only five independent real attack sessions, and no recording of a benign planned grandmaster change — the one confounder we would most like to have. Real-world false-alarm rate on production timing hardware is therefore not established.

Cards A/B/C for simulator, TIMESAFE captures, live netem runs — plus the honest-limit band.

WHAT TO SAY

- State plainly which numbers you trust. The simulator is a sandbox; TIMESAFE is the evidence.
- Volunteer the five-session limit before anyone asks. Doing so is what makes 99.96% read as credible rather than suspicious.
- TIMESAFE is public data from a real 5G testbed at Northeastern University — say 'published with their research' so nobody thinks you fabricated it.

TERMS ON THIS SLIDE

Synthetic vs real

Synthetic means your own physics simulator generated it with perfectly known labels. Real means captured from actual equipment.

TIMESAFE

The public dataset of real PTP attack captures from Northeastern University's 5G testbed. Five independent sessions of Announce and Sync attacks, each with a benign baseline.

netem

A Linux kernel facility for injecting network impairments — delay, jitter, packet loss — on a virtual link. Used to stress two real ptp4l daemons.

ptp4l

The standard open-source Linux PTP daemon. Running against it is what makes the live results 'real software' rather than simulation.

Benign planned grandmaster change

A legitimate, scheduled switch of the master clock. The single most valuable confounder to have a recording of — because it looks like an attack and is not. You do not have one.

IF ASKED

Only five sessions — isn't that too few? Yes, and it is on the slide. It is why the deployment false-positive rate is not established and why Tier-3 is the next phase. Do not defend the sample size; concede it and redirect to the roadmap.

RESULTS — THE EVIDENCE THAT COUNTS

Results on real TIMESAFE attack captures

"Protection" = the attack was classified H1 or safely routed to UNKNOWN, after 2-of-3 persistence.

99.96%

Real Announce / BMCA attack
100% of episodes detected within 2 s

99.66%

Real Sync / Follow-Up attack
100% of episodes detected within 2 s

99.66%

Real single-step Sync attack
100% of episodes detected within 2 s

Benign false alarms on real captures

4.49% → 2.37%

Cut by the 2-of-3 persistence rule, at -0.4 s added latency and no loss against the 2-second deadline.

Unseen healthy-looking GNSS spoof

0.00%

A known theoretical limit, not a coding defect. Detail on the negative-results slide.

What made the real numbers move: adding BMCA grandmaster-transition features raised real Announce protection from **23.9%** to **approximately 100%**. Group-wise open-set recovered unseen DoS from 0% to approximately 94%.

Source: ORAN_Project_Results.xlsx · 5_Real_and_Live; README.md 'Final validated results'

Three headline protection figures, the false-alarm reduction, and the 0.00% limit card.

WHAT TO SAY

- Lead with the sentence that shows the result came from an insight, not from tuning: adding BMCA grandmaster-transition features raised real Announce protection from 23.9% to approximately 100%.
- Then the three numbers. Then the false-alarm reduction. Then the red card — introduce the 0.00% yourself rather than letting someone find it.
- Say what 'protection' means before quoting any percentage.

TERMS ON THIS SLIDE

Protection

Your defined metric: the attack was either classified H1 or safely routed to UNKNOWN, after 2-of-3 persistence. Note that safely-routed-to-UNKNOWN counts as a success — be upfront about that, it is a generous definition and you should own it.

Announce / BMCA attack

An attacker forges Announce messages claiming excellent clock quality, wins the Best Master Clock Algorithm election, and becomes the time source for everything downstream. Attack family 1.

Sync / Follow-Up attack

Manipulation of the two-step PTP messages that carry the master's transmit timestamp.

Single-step Sync

The one-step PTP variant where the timestamp is embedded in the Sync message itself rather than in a following message.

Benign false alarm

A window of normal, healthy traffic that the system wrongly flagged. 4.49% before the voting rule, 2.37% after.

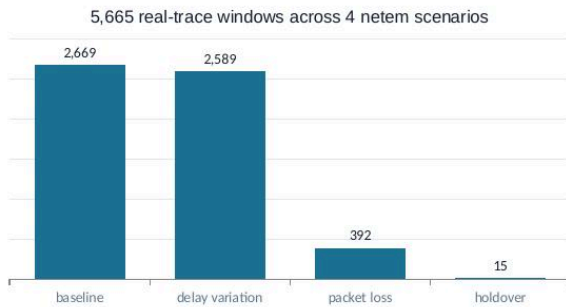
IF ASKED

Isn't counting UNKNOWN as success too generous? It is a defensible definition because UNKNOWN routes to a conservative action that does not make the outage worse. But say so explicitly rather than hiding it — a reviewer who works it out unaided will distrust the rest.

Why is Announce better than Sync? The BMCA feature group was specifically engineered for grandmaster-election behaviour, which is what an Announce attack disturbs.

LIVE VALIDATION

It runs on real linuxptp, not only on recordings

**0.081 s**Mean live decision latency
(worst observed 0.404 s)**14.68 ns**pcap parser offset-recovery
error vs known ground truth **$r = 0.998$** Digital twin vs simulator
agreement on action ranking**87.60 min**Longest uninterrupted normal
session — 2 h target not met

Fail-closed, proved on real software. We synchronised two real ptp4l daemons, then cut the link and killed the master. Across the 48 consecutive windows that followed: **47 UNKNOWN + 1 pending**, zero windows wrongly labelled healthy.

Feature coverage on live data: live management data moved 10 of 28 features; packet captures moved 14 of 28. SyncE quality and physical GNSS status still require real equipment.

Source: TEAM_REPORT.md 'What happened live' · ORAN_Project_Results.xlsx · 5_Real_and_Live

Window-count chart, four stat cards, the fail-closed proof band, feature-coverage note.

WHAT TO SAY

- This slide answers 'does it only work on your own data?'. No — two real ptp4l daemons, real Linux impairments.
- The holdover bar is tiny (15 windows) because that scenario ends fast by design. Say so before anyone reads it as a failure.
- The 87.60 minutes is in amber deliberately: you targeted two hours, the run was interrupted, you are not claiming it.
- Finish on the fail-closed result — 47 UNKNOWN + 1 pending, zero windows wrongly called healthy.

TERMS ON THIS SLIDE

Decision latency

Time from telemetry arriving to an action being chosen. 0.081 s mean, 0.404 s worst — roughly 25x inside the 2 s budget.

pcap

A packet-capture file. 14.68 ns is the error in recovering clock offset from a capture with known ground truth — proof the parser is correct to the nanosecond.

Pearson $r = 0.998$

Twin-vs-simulator agreement on how it ranks candidate actions.

10 of 28 / 14 of 28 features

How many of the 28 features actually varied on live data. The rest need equipment you do not have — SyncE quality and physical GNSS status. State this; it bounds what the live result proves.

48 consecutive windows

The windows captured immediately after the master clock was killed. 47 went to UNKNOWN, 1 was pending the 2-of-3 vote, none were called healthy.

IF ASKED

Why did the 2-hour run fail? It was interrupted. 87.60 minutes is what you have and what you claim. Longer continuous baselines are explicitly in the Tier-3 roadmap.

SIMULATOR RESULTS & BASELINES

Does the governed loop beat doing something simpler?

Simulator, averaged over 8 random seeds. These prove the method is sound — they are not field performance.

Discriminator & healing loop

Discriminator accuracy	0.991 ± 0.002
Macro F1	0.991 ± 0.002
ROC-AUC (H1)	1.000 ± 0.000
Recovery success	1.000 ± 0.000
Wrong-action rate	0.000 ± 0.000
Mean time to recovery	0.933 ± 0.008 s

No neural networks, no GPU. Random Forest + Isolation Forest, chosen for interpretability, data scale and exact reproducibility.

Method	Recovery	Wrong-action	MTTR (s)	Peak error (ns)	Within 2 s
Governed loop (ours)	1.00	0.00	0.935	43.5	100%
Always failover	0.50	0.00	0.635	91.5	100%
Always holdover	0.50	0.50	1.175	91.5	100%
Detection only (no fix)	0.00	0.00	2.000	98.9	0%

Baseline comparison — benchmark_results.csv

Reading the table. Always-holdover recovers half the time and applies the **opposite** fix in half of all cases. Detection-only never recovers and misses the deadline every time. Only the governed loop reaches full recovery with zero wrong actions — which is the argument for discrimination over alarming.

Source: ORAN_Project_Results.xlsx · 3_Results_Core

Six-metric summary panel plus a four-row baseline comparison table.

WHAT TO SAY

- Open with the qualifier on the slide: simulator, 8 seeds, method soundness — not field performance.
- The sharpest row is always-holdover: it applies the OPPOSITE fix in half of all cases. That single number is the argument for discrimination over alarming.
- Detection-only never recovers and misses the deadline every time — that is the 'just raise an alarm' strategy, quantified.

TERMS ON THIS SLIDE

Always failover / always holdover

Deliberately dumb comparison strategies: apply one fixed response regardless of what happened.

Detection only

Raise an alarm, apply no fix. The status quo in most deployments.

Wrong-action rate

How often the system applied the opposite of the correct fix. 0.50 for always-holdover because half the events are attacks, where coasting is wrong.

Peak error (ns)

Worst clock error reached during recovery. Lower is better; 43.5 ns for the governed loop versus 91.5 ns for the naive strategies.

No neural networks, no GPU

A deliberate choice. Tree ensembles dominate on small tabular datasets, everything trains in seconds on CPU with fixed seeds, and feature importances stay inspectable.

IF ASKED

Why not deep learning? Thousands of windows with 28 tabular features is exactly the regime where tree ensembles beat neural networks — this is a well-replicated empirical result, not a preference. Add: this is a safety decision path that needs inspectable feature importances, and the claimed contribution is integration and validation, not a new learning algorithm.

Perfect recovery and zero wrong actions — really? On the simulator, with perfectly known labels, yes. Pivot immediately to the real TIMESAFE numbers, which are not perfect. Imperfect numbers on real data are stronger evidence than perfect numbers on synthetic data, and saying so yourself is what buys credibility.

How reliable are the feature importances? Check whether you used impurity-based importance (scikit-learn's default) or permutation importance. Impurity-based importance is biased toward continuous, high-cardinality features — which would bias AGAINST your integer-count BMCA features, meaning the real effect is likely at least as large as reported. Say which one you used.

ENGINEERING HONESTY

A real safety defect, found and closed

1 Zero looked like perfect sync

Under heavy packet loss the timing tool returned nothing, our software substituted a zero — and a zero is indistinguishable from perfect synchronisation. The system reported "healthy" at the exact moment it had gone blind, and skipped the novelty layer on the way. An attacker could trigger this deliberately by flooding the link.

2 Stale numbers were worse than zero

We then killed a real ptp4l master. The tool did not report an error or a zero — it kept confidently reporting the last numbers it had seen, indefinitely. Only the port connection state told the truth. Anything trusting the numbers alone would have believed the network was fine throughout a total blackout.

Situation	Before	After
No timing data received	"healthy"	UNKNOWN → safe response
Half the samples missing	"healthy"	UNKNOWN → safe response
Data with no proof of origin	"healthy"	UNKNOWN → safe response
Twin confidence with no data	100%	15% (correctly distrusted)
Normal healthy traffic	"healthy"	"healthy" (unchanged)

The fix, and why it holds

Missing data is now its own state, never a number.

Every window carries provenance (telemetry_valid, valid_sample_fraction), and nothing is declared healthy unless the system can prove it saw the data.

Ten tests were written before the fix and confirmed to fail against the old code — they test the bug, not the patch. Detection accuracy was unaffected (99.1%); the suite grew to 56 tests.

Two problem cards, a before/after table, and the fix panel.

WHAT TO SAY

- Do not skip this to save time. Finding, reproducing and closing a fail-open defect — with tests written BEFORE the fix — is stronger evidence of engineering discipline than any accuracy number in the deck.
- Tell it as a sequence: zero looked like perfect sync; then we killed a real master and found something worse, stale numbers reported confidently forever.
- Close on the tests: ten tests written before the fix and confirmed to fail against the old code. They test the bug, not the patch.

TERMS ON THIS SLIDE

Fail-open vs fail-closed

Fail-open means defaulting to 'everything is fine' when you lose visibility. Fail-closed means refusing to certify healthy without proof. The bug was that the system failed open.

Why zero is dangerous

A clock offset of zero means perfect synchronisation. Substituting zero for missing data makes total blindness look like perfection — and an attacker can trigger it deliberately by flooding the link.

telemetry_valid /
valid_sample_fraction

Provenance flags carried on every window recording whether each underlying measurement was genuinely observed. These are what make the fail-closed gate possible.

Twin confidence 100% -> 15%

With no input data the twin previously reported maximum confidence, which switched off the safeguard that forces a cautious response. It now correctly distrusts itself.

Port connection state

The one field that told the truth when the master was killed. The numeric readings kept reporting stale values indefinitely; the connection state did not.

IF ASKED

How do you know the fix works? Ten tests written before the fix and verified to fail against the old code — so they test the actual defect rather than the patch. Then proved end-to-end on real ptp4l by cutting the link and killing the master.

Did the fix cost accuracy? No. Detection accuracy was unaffected at 99.1%; the suite grew from 46 to 56 tests.

NEGATIVE RESULTS

What did not work, and the limit software cannot cross



Unseen healthy-looking GNSS spoof

0.00% protection

When the family is strictly excluded from training, a spoof that behaves exactly like healthy timing is in-distribution from a single reference. Not a coding defect.



Cross-source agreement features

0.00 pp gain

Seven experimental features comparing GNSS, network PTP and a peer source added no combined protection and regressed established families. Gated off by default; stats/multisource_eval.py reproduces the comparison.



Trusting reported GNSS status

Attacker-influenced

A spoofed receiver can still report that it is synchronised. Receiver-reported state cannot be trusted on its own.



Closed-set transfer across attack styles

0% recall

An Announce-trained closed-set classifier tested on Sync sessions has zero recall. A model trained on one real attack style can fail completely on another.

The standing limit is physical, not algorithmic. Software can detect protocol manipulation, unfamiliar traffic, jamming and many physics inconsistencies. It cannot prove a single healthy-looking clock is telling the truth with no independent trusted reference. Solving it requires something the attacker cannot forge — authenticated GNSS (e.g. Galileo OSNMA), a physically independent clock, or hardware cross-checking.

Four negative-result rows plus the standing-limit band.

WHAT TO SAY

- Present the 0.00% as a result, not an apology. Identifying precisely where a software-only approach stops is the strongest argument in the deck for continuing the project.
- The cross-source result is the one to dwell on: seven features built, measured, found to add nothing, and switched off. That is discipline, not failure.
- End on the limit statement — it sets up the Tier-3 ask on the next slide.

TERMS ON THIS SLIDE

In-distribution

Statistically indistinguishable from the healthy data the model was trained on. A perfect spoof is in-distribution from a single reference, which is why no amount of feature engineering catches it.

0.00 pp gain

Percentage points. The cross-source features added literally nothing to combined protection, and made some established results worse.

Coherent compromise

If every time reference is attacked in the same way, they all still agree with each other — so comparing them tells you nothing.

Attacker-influenced

A spoofed GNSS receiver can still report that it is synchronised. Its self-reported status cannot be trusted on its own.

Galileo OSNMA

Open Service Navigation Message Authentication — cryptographically signed satellite navigation messages, so a receiver can verify the signal is genuine. One of the few things an attacker cannot forge.

Closed-set transfer

A classifier trained on Announce attacks, tested on Sync attacks, gets 0% recall. One real attack style does not transfer to another.

IF ASKED

Isn't 0% a failure? It is a measured physical limit, not a defect. Detecting a perfect spoof from a single reference is information-theoretically impossible — you need independently trustworthy evidence. That is precisely what Tier-3 hardware provides.

Why keep the cross-source code if it does nothing? Reproducibility. `stats/multisource_eval.py` regenerates the negative result on demand, so the finding is checkable rather than asserted.

STATUS & ROADMAP

Where the project stands, and the next justified phase

Software build

Complete

56 tests passing; one command reproduces every number

Software validation

Complete

Validated on real recorded captures and live real ptp4l

The safety defect

Closed

Found, fixed, hardened and verified on real PTP software

Hardware (Tier-3)

Not started

Timing card, GNSS grandmaster, authenticated GNSS receiver

Evasion-attack study

Designed

Software-only research add-on; being built separately

Decision. Software feature work is closed — further single-source features now give diminishing returns. Build was largely AI-generated under human direction, offset by 56 tests, a mechanical repository audit, and tests written before fixes.

Tier-3 hardware validation

- Authenticated GNSS and physically independent clocks
- Hardware timestamping NIC; real O-DU / O-RU in the path
- Longer continuous normal-operation recordings
- Controlled authorised attacks with a benign planned grandmaster change

Evasion-attack study (research novelty)

Stop only defending: design a timing attack that slips past our own detector while still breaking the clock. No hardware, reuses the existing stack, and turns the story into "new attack → our defence stops it".

Restore point: `git reset --hard known-good-2026-08-07` · Application path: vendor-neutral timing-security validation for telecom labs, private 5G operators and equipment vendors

Five status rows, the Tier-3 list, the evasion-attack panel, and the decision note.

WHAT TO SAY

- Close on the decision: software feature work is closed because you MEASURED the point of diminishing returns rather than guessing at it.
- The ask is Tier-3 hardware. The evasion study runs in parallel and needs nothing.
- The AI-generation line is on the slide because your own walkthrough and technical report both document it. If this room is the wrong audience for that, cut the sentence — but do not let them discover it elsewhere.

TERMS ON THIS SLIDE

Tier-3

Your third validation tier: real hardware. Authenticated GNSS, physically independent clocks, hardware timestamping NIC, real O-DU/O-RU in the path.

Hardware timestamping NIC

A network card that timestamps packets in silicon rather than in software. Software timestamps on a laptop resolve in microseconds; the target regime is ~100 nanoseconds. This is why almost every normal live window was flagged unfamiliar.

Evasion attack

Reversing roles: instead of only defending, design a timing attack shaped to slip past your own detector while still breaking the clock. Turns the paper narrative into 'new attack, our defence stops it'.

known-good-2026-08-07

A git tag marking the last fully verified state. One command rolls the whole repository back to it.

IF ASKED

Why stop adding features? Because the returns were measured, not assumed. The cross-source experiment added 0.00 pp and regressed established families — that is the evidence that single-source feature engineering is exhausted.

How much of this was AI-written? Answer directly: most of the build, under human direction, offset by 56 tests, a mechanical repository audit (scripts/audit_state.py) that catches overstated completion, and tests written before fixes. The verification regime is the answer, not a denial.

Appendix

Leave-one-out generalisation · full protection table · the 28 features · reproduction · honest limitations

Section break.

WHAT TO SAY

- Jump here for questions. Most review questions land on generalisation (slide 14) or the feature list (slide 16).

APPENDIX A • GENERALISATION

Can it catch an attack family it never saw?

Remove one whole attack type from training, then test on it. This is the honest, hard test — and it is what motivated every layer added after the plain classifier.

Held-out family (never trained on)	Recall — plain classifier	Reading
Spoof	1.000	Caught perfectly even when unseen.
Replay	0.592	Partially caught.
DoS / message flood	0.000	Missed by the plain classifier — this is why the open-set layer exists.
GNSS spoof	0.000	The fundamental limit: a healthy-looking spoof is invisible from one reference.
GNSS jam	0.000	Plain classifier misses it; oscillator-consistency features recover it.

These gaps are the design rationale, not a weakness. The open-set novelty layer, the BMCA transition features and the oscillator-consistency features were each added to close a specific row in this table. Group-wise open set recovered unseen DoS from 0% to approximately 94%; oscillator consistency solved unseen GNSS jamming. Only the healthy-looking GNSS spoof survives, and that one is physical.

Source: ORAN_Project_Results.xlsx • 4_Generalization

Leave-one-out recall table for the plain classifier, plus the design-rationale band.

WHAT TO SAY

- If a mentor asks 'how do you know it generalises', this is the slide.
- The answer is: it did not, we measured exactly where, and each layer we added targets a named row in this table.
- Three zeros in this table are not embarrassing — they are the reason the architecture has the shape it has.

TERMS ON THIS SLIDE

Leave-one-out

Remove one entire attack family from training, then test only on that family. The honest, hard generalisation test — much harder than a random train/test split.

Recall

Of all the actual attacks present, what fraction did the system catch. Different from accuracy, which also counts correct rejections.

Plain classifier

The Random Forest alone, without the open-set novelty layer, BMCA features or oscillator-consistency features. These are the numbers BEFORE the fixes.

0.592 for replay

Partially caught. Replay is anomalous in timing space but ordinary in BMCA space, which is exactly the case group-wise open-set detection was built for.

IF ASKED

Three zeros looks bad. Those are the plain classifier's numbers and the reason the extra layers exist. Group-wise open-set recovered unseen DoS from 0% to about 94%; oscillator consistency solved unseen GNSS jamming. Only the healthy-looking GNSS spoof survives, and that one is physical.

APPENDIX B • FULL RESULTS

Final validated results, every domain

Combined protection = H1 classification or conservative UNKNOWN routing, after 2-of-3 persistence.

Domain	Held-out family	Protection	Episodes within 2 s
Simulator	Spoof	93.30%	100%
Simulator	Replay	80.45%	100%
Simulator	DoS / message flooding	88.27%	100%
Simulator	GNSS jam	91.57%	100%
Simulator	Strict unseen GNSS spoof	0.00%	0%
TIMESAFE (real)	Announce / BMCA	99.96%	100%
TIMESAFE (real)	Sync / Follow-Up	99.66%	100%
TIMESAFE (real)	Single-step Sync	99.66%	100%

2.37%

Benign false alarms after 2-of-3 persistence, down from 4.49% at 1-of-1

Test inventory

- 56 automated tests across 15 files
- 10 lock in the fail-closed behaviour
- Real pmc captures used in the safety tests
- audit_state.py mechanically truth-checks the repository

Note on the simulator column. Sandbox measurements with perfectly known labels. Where simulator and real numbers disagree, the real ones govern — and the multi-source experiment kept related GNSS attacks in training, so its higher single-source result does not overturn the strict 0% unseen-spoof finding.

Source: README.md 'Final validated results' • TEAM_REPORT.md 'Final measured results'

Eight-row protection table across simulator and TIMESAFE, plus false alarms and test inventory.

WHAT TO SAY

- Keep this slide ready — it is the one you will be asked to put back up.
- The rows are grouped deliberately: simulator first (sandbox), TIMESAFE last (evidence). If simulator and real numbers ever disagree, the real ones govern.

TERMS ON THIS SLIDE

Combined protection

H1 classification OR conservative UNKNOWN routing, after 2-of-3 persistence. Both count as protected.

Episodes within 2 s

Of the attack episodes that were caught, the fraction detected inside the failure window. 100% everywhere except the unseen GNSS spoof, which is 0% because nothing was detected at all.

audit_state.py

A script that mechanically truth-checks the repository against its own claims — it exists specifically to catch AI coding agents overstating completion.

Real pmc captures

pmc is the linuxptp management client. Its real captured output is used inside the safety tests, so the fail-closed behaviour is tested against genuine tool output.

IF ASKED

Why are simulator numbers lower than real numbers? Different tests. The simulator rows are strict leave-one-out on unseen families; the TIMESAFE rows are leakage-controlled evaluation on real attack traffic with sessions kept separate. They are not comparable head to head — say so rather than letting the contrast look odd.

APPENDIX C • FEATURE DESIGN

The 28 shipped features, in six groups

5 Timing

Offset, delay and jitter — the clock error itself

offset_mean
offset_std
offset_abs_max
path_delay_mean
pdv_std

2 Protocol regularity

Whether PTP messages arrive in the expected order and rhythm

seq_regressions
msg_irregularity

2 Message rate

How many messages per second, and how steady that rate is

msg_rate_mean
msg_rate_std

7 BMCA / grandmaster

Clock-election behaviour — raised real Announce protection
23.9% → ~100%

gm_identity_changes
gm_identity_churn
clock_class_changes
clock_class_improve_jump
priority1_changes
steps_removed_changes
steps_removed_min

8 Time source

What the GNSS / SyncE receiver reports about its own state

sync_e_ql_max
gnss_loss_rate
holdover_rate
gnss_status_changes
antenna_fault_rate
satellites_drop_max
satellites_mean
holdover_entry_count

4 Oscillator consistency

Whether physical oscillator behaviour matches the declared state

drift_vs_declared_state_residual
holdover_spec_violation_rate
status_behaviour_disagreement
offset_step_vs_drift_ratio

A deliberate exclusion. Raw grandmaster identities and MAC addresses are never used as features — only relative signals. A raw identity would let the model fingerprint the capture file rather than learn the attack. Seven cross-source features exist but ship disabled.

Source: ORAN_SPlane_Technical_Report.pdf §3.4

Six cards, each with its feature count, what it measures, and every feature name.

WHAT TO SAY

- Do not read 28 names aloud. Name the six groups and move on.
- The one thing worth saying: the BMCA group is what raised real Announce protection from 23.9% to about 100%.
- Volunteer the leakage defence — raw grandmaster identities and MAC addresses were deliberately excluded.

TERMS ON THIS SLIDE

BMCA

Best Master Clock Algorithm. Devices advertise clock quality in Announce messages (priority1, clockClass, clockAccuracy, stepsRemoved) and the best one wins the election. Automatic and unauthenticated in the base standard — so an attacker who simply claims to be an excellent clock wins.

gm_identity_churn

How often the elected grandmaster identity keeps changing. Rapid churn is an election being fought, not a stable network.

clock_class_improve_jump

A device suddenly claiming a much better clock class than before. Legitimate clocks rarely improve abruptly; attackers do.

steps_removed

How many hops the device is from the grandmaster. Sudden changes indicate the timing tree was rearranged.

pdv_std

Packet Delay Variation standard deviation — how much the network delay jitters.

SyncE QL

Synchronous Ethernet Quality Level, advertised in ESMC messages. SyncE distributes frequency (tick rate) only, not time of day.

Oscillator consistency

Cross-checking the physical crystal's actual drift against what the receiver claims its state is. Catches a receiver that is lying about being healthy.

Why raw identities are excluded

A raw grandmaster ID or MAC address is a high-cardinality categorical. The model would learn which capture file it is reading rather than what an attack looks like. Deliberate leakage defence.

IF ASKED

Why 28 and not more? Seven additional cross-source features exist and ship disabled because they measured 0.00 pp gain and regressed established families.

APPENDIX D · REPRODUCTION

Reproducing every number in this deck

Step	Command	Produces
1 · Setup	<code>python -m venv .venv && pip install -r requirements.txt</code>	The environment
2 · Tier-1 pipeline	<code>python scripts/run_all.py</code>	<code>results/SUMMARY.md</code> — core numbers, ~1 min
3 · Tier-2 suite	<code>python scripts/run_tier2.py</code>	<code>results/tier2/TIER2_REPORT.md</code> — CIs, generalisation, real ingest, ~3 min
4 · All tests	<code>python -m pytest tests -q</code>	56 tests pass, ~5 min
5 · Live (Linux + root)	<code>sudo python3 -m harness.run_netem_scenarios --scenarios baseline pdv loss holdover --duration 90</code>	<code>results/tier2/netem/</code> real-trace windows
6 · Verify repo truth	<code>python scripts/audit_state.py</code>	Pass/fail truth-check of the whole repository
7 · Restore if broken	<code>git reset --hard known-good-2026-08-07</code>	Rolls back to the verified-good commit

CPU-only, no GPU, runs on a normal laptop. Fixed seeds throughout, so any result regenerates exactly. Result CSVs are already on disk under `results/` and `results/tier2/` — every number in this deck traces to one of them.

Source: `ORAN_Project_Results.xlsx · 6_Reproduce`

Seven-step command table plus the CPU-only note.

WHAT TO SAY

- Offer to run steps 2 and 3 live if there is any doubt. Together they take about four minutes.
- The point of this slide is not the commands — it is that a stranger with the repository can regenerate every figure you just showed.

TERMS ON THIS SLIDE

Tier-1 / Tier-2

Your validation tiers. Tier-1 is the core simulator pipeline; Tier-2 adds confidence intervals, generalisation tests and real-data ingest. Tier-3 is hardware and has not started.

Fixed seeds

The random number generator is pinned, so results regenerate exactly rather than approximately.

`audit_state.py`

Truth-checks the repository against its own claims. Run it if anyone doubts the completion status.

IF ASKED

Can we run it ourselves? Yes. Steps 1-4 need no hardware and no GPU, and take about nine minutes end to end.

APPENDIX E · HONEST LIMITATIONS

What we are explicitly not claiming

- No hardware timestamping NIC, physical clock, authenticated GNSS receiver, O-DU or O-RU has been validated.
- Only five independent real attack sessions, and no benign planned-grandmaster-change capture — so deployment false positives may be higher than 2.37%.
- SyncE quality requires a live sync4l feed or O-RU M-plane telemetry and cannot be recovered from ordinary pcaps.
- An Announce-trained closed-set classifier tested on Sync sessions has 0% recall — one real attack style does not transfer to another.
- Almost every normal software-timestamped live window was flagged unfamiliar, because laptop/WSL timing resolves in microseconds, not the ~100 ns hardware target.
- The continuous baseline reached 87.60 minutes, not the intended two-hour target.
- These are research measurements, not telecom certification.

Stating these first is what makes the rest of the deck load-bearing.

Seven enumerated limitations.

WHAT TO SAY

- Read two or three aloud, not all seven. The point is that you enumerated them before anyone asked.
- Close on the last line: stating these first is what makes the rest of the deck load-bearing.

TERMS ON THIS SLIDE

Why every live window looked unfamiliar

Laptop/WSL software timestamping resolves in microseconds. The detector is calibrated for a ~100 ns hardware regime, so ordinary laptop timing looks abnormal to it. This is a measurement-environment limitation, not a model defect — and it is exactly what a hardware timestamping NIC fixes.

sync4l

The Linux daemon that reports SyncE quality. Without it running, or without O-RU management-plane telemetry, SyncE quality cannot be recovered from ordinary packet captures.

Not telecom certification

These are research measurements produced by a reproducible prototype. They are not conformance testing against a standard.

IF ASKED

So what would change your conclusions? Hardware. Authenticated GNSS and a physically independent clock would attack the one problem software provably cannot solve, and a hardware timestamping NIC would establish the real false-alarm rate.